

UNIT 3

Design and Analysis of Machine Learning Experiments

Design of machine learning experiments to assess and compare the performances of learning algorithms in practice and the statistical tests to analyze the results of these experiments

Design and Analysis of Machine Learning Experiments

- We explored various learning algorithms and observed that multiple algorithms can be suitable for a given application. Now, we focus on **two key questions**
 - **How can we assess the expected error of a learning algorithm on a problem?**
 - For instance, after training a classifier on a dataset from a specific application, can we confidently predict that its error rate in real-world use will be below a certain threshold, such as 2%?
 - Suppose we train a spam detection model using a classification algorithm on a dataset of emails. To deploy it in real-world scenarios, we need to estimate its expected error rate. If our evaluation methods (e.g., cross-validation, test set analysis) indicate that the model misclassifies less than 2% of emails, we can be reasonably confident that its real-world error rate will also stay within this limit.

Design and Analysis of Machine Learning Experiments

- Given two learning algorithms, how can we say one has less error than the other one, for a given application?
 - The algorithms being compared may differ in type, such as parametric versus nonparametric, or in their hyperparameter settings.
 - For instance, when evaluating a multilayer perceptron with four hidden units versus one with eight, we need a method to identify which has a lower expected error. Similarly, in the case of the k-nearest neighbor classifier, we must determine the optimal value of k for better performance.

Guidelines for Machine Learning Experiments

- **Aim of the Study**
- **Selection of the Response Variable**
- **Choice of Factors and Levels**
- **Choice of Experimental Design**
- **Performing the Experiment**
- **Statistical Analysis of the Data**

Aim of the Study

- Clearly define the problem and objectives.
- Common goals include estimating a learning algorithm's expected error and ensuring it meets an acceptable threshold.
- Given two algorithms and a dataset, we aim to determine which has lower generalization error.
- Comparisons can involve:
 - Different algorithms
 - Variants of the same algorithm (e.g., improved feature extraction)
- In broader cases, we may:
 - Compare multiple algorithms and rank them by error.
 - Evaluate algorithms across multiple datasets for a comprehensive assessment.

Selection of the Response Variable

Common Measures

- Classification: Misclassification error (0/1 loss)
- Regression: Mean Squared Error (MSE)

Alternative Metrics

- Risk measures (generalized loss functions)
- Information Retrieval: Precision, Recall
- Cost-Sensitive Learning: Considers both output accuracy and system complexity.

Choice of Factors and Levels

Factors depend on study objectives

- Hyperparameter tuning: Factors are the algorithm's parameters.
- Algorithm comparison: The learning algorithm itself is a factor.
- Dataset variations: Different datasets become a factor.

Choosing Factor Levels

- Carefully select levels to avoid missing optimal configurations or unnecessary experiments.
- Normalize factor levels when needed (e.g., scale-dependent parameters like Parzen window spread).

Best Practices:

- Use domain expertise but remain open to exploring all relevant factors.

Choice of Experimental Design

Factorial Design:

- Preferred unless factors are proven to be independent.
- Most factors interact, so factorial analysis ensures better insights.

Replication & Dataset Size:

- Large datasets require fewer replications.
- Too few replicates lead to unreliable comparisons and invalid statistical tests.

Train-Test Splitting & Resampling:

- Reserve a portion of data for testing, using resampling for training and validation.
- Poor splitting can lead to high variance and inconclusive results.

Choice of Datasets:

- Avoid synthetic and toy datasets; prefer real-world data.
- Simple datasets help with intuition but may not reflect real-world complexity in high-dimensional spaces

Performing the Experiment

Preliminary Checks:

- Conduct trial runs with random settings before large experiments.
- Save intermediate results or random seeds for reproducibility.

Experimental Integrity:

- Be aware of software aging effects in long experiments.
- Maintain unbiased evaluation, especially when comparing algorithms.
- Ideally, separate developers from testers in large-scale studies.

Reliable Code & Documentation:

- Use well-tested libraries instead of writing custom implementations.
- Maintain clear documentation, especially for collaborative work.
- Apply software engineering best practices to ensure experiment quality.

Statistical Analysis of the Data

Ensuring Reliable Conclusions:

- Avoid subjective or chance-based interpretations.
- Use hypothesis testing to validate claims.

Example:

- Instead of asking, "Is Algorithm A more accurate than B?"
- Formulate as: "Is the average error of A significantly lower than B?"

Visualization Techniques:

- Histograms, box plots, range plots help analyze error distributions.

Subject	t	Feature 1	Feature 2	Target		Subject	t	Feature 1	Feature 2	Target
Paul	1	1000	male	0	Train	Paul	1	1000	male	0
Paul	2	1100	male	0		Paulina	1	10000	female	0
Paul	3	1200	male	1		George	1	50000	male	1
Paul	4	1300	male	1		Paul	2	1100	male	0
Crista	4	20	female	0		Paulina	2	100000	female	1
Crista	5	100	female	0		George	2	50000	male	1
Paulina	1	10000	female	0		Paul	3	1200	male	1
Paulina	2	100000	female	1		Paulina	3	95000	female	1
Paulina	3	95000	female	1		George	3	50000	male	1
Paulina	4	97000	female	1		Paul	4	1300	male	1
Paulina	5	99000	female	1	Test	Crista	4	20	female	0
Paulina	6	101000	female	1		Paulina	4	97000	female	1
George	1	50000	male	1		George	4	50000	male	1
George	2	50000	male	1		Crista	5	100	female	0
George	3	50000	male	1		Paulina	5	99000	female	1
George	4	50000	male	1		George	5	50000	male	1
George	5	50000	male	1		Paulina	6	101000	female	1
George	6	50000	male	1		George	6	50000	male	1

x		y
1	2	0
3	4	1
5	6	1
7	8	0
9	10	1
11	12	0
13	14	0
15	16	1
17	18	1
19	20	0
21	22	1
23	24	0

x		y	Training set
17	18	1	
5	6	1	
23	24	0	
1	2	0	
3	4	1	
11	12	0	
15	16	1	
21	22	1	
7	8	0	
			Test set
9	10	1	
13	14	0	
19	20	0	

Cross-Validation (CV)

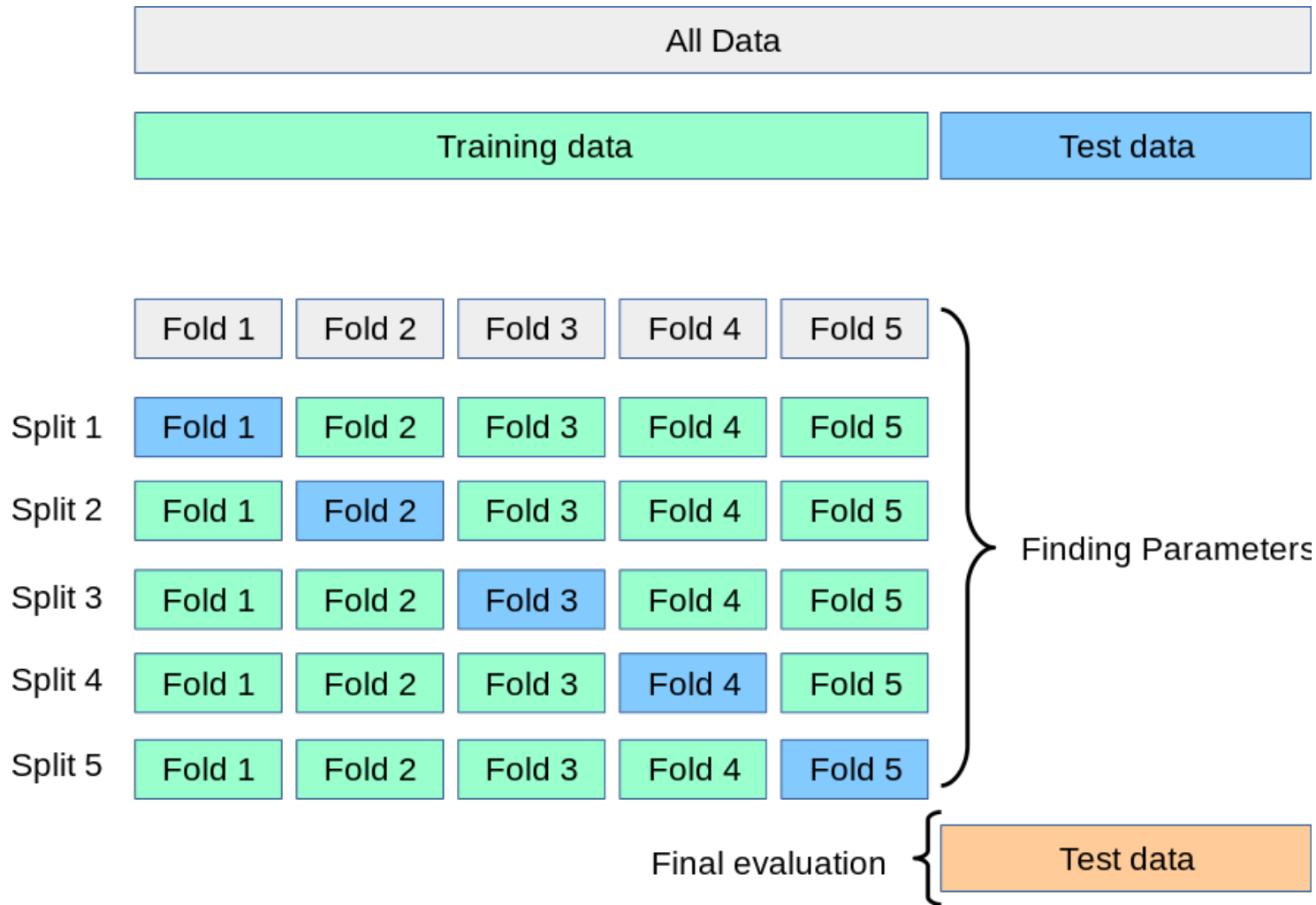
- Cross-validation is a technique used in machine learning to assess the performance and generalizability of a model by splitting the dataset into multiple subsets.
- It helps prevent overfitting and ensures that the model performs well on unseen data.

Types of Cross-Validation

- **Holdout Validation:** The dataset is split into training and testing sets (e.g., 80% training, 20% testing).
- **k-Fold Cross-Validation:** The dataset is divided into k equal-sized folds. The model is trained on $k-1$ folds and tested on the remaining one. This is repeated k times, and the results are averaged.
- **Stratified k-Fold Cross-Validation:** Similar to k-Fold but ensures that class distributions are maintained in each fold (useful for imbalanced datasets).
- **Leave-One-Out Cross-Validation (LOOCV):** A special case of k-Fold where k is equal to the number of samples, meaning each sample is used as a test set once.
- **Leave-P-Out Cross-Validation:** p samples are left out for validation while the remaining are used for training.
- **Time Series Cross-Validation:** Used for time-dependent data, where past data is used for training and future data for testing.

k-Fold Cross-Validation

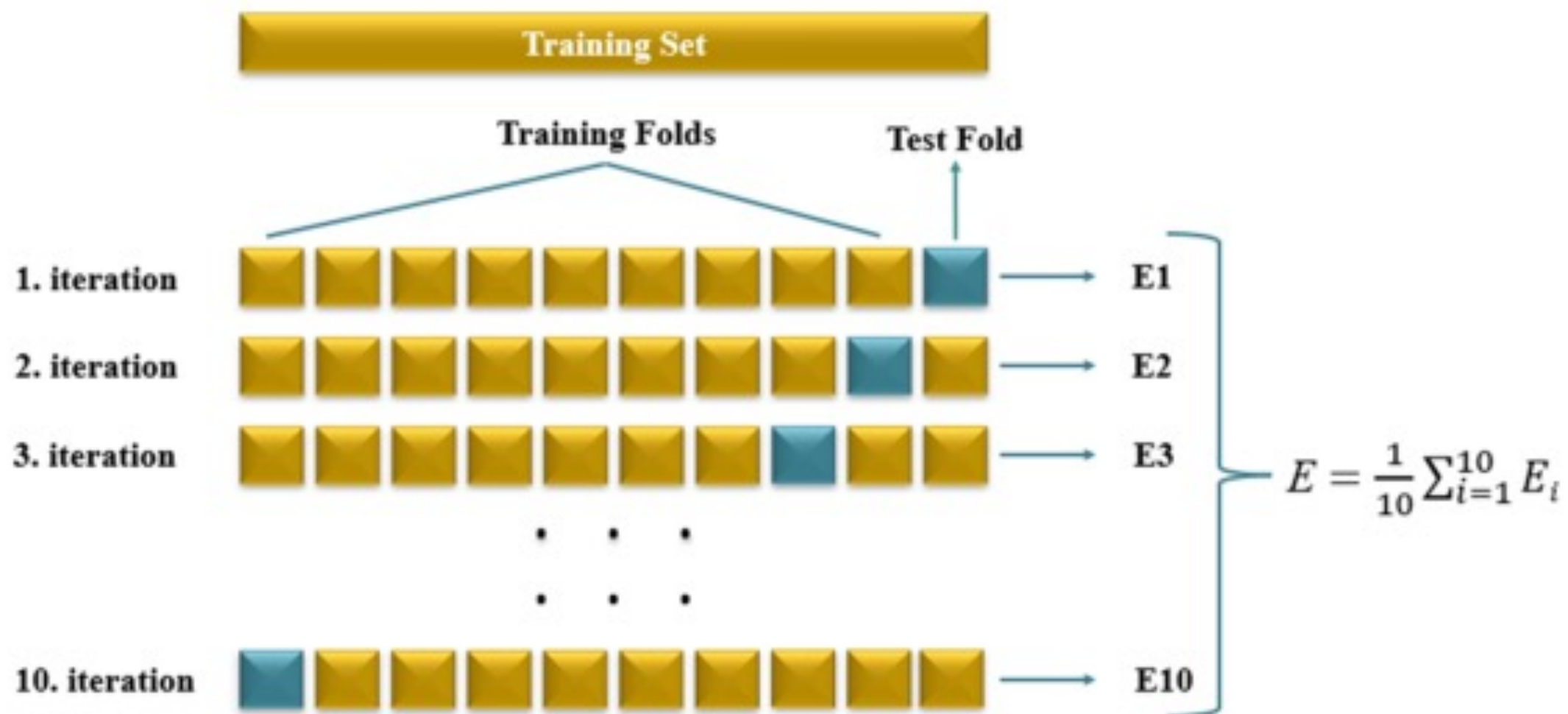
- k-Fold Cross-Validation is a technique used to evaluate the performance of a machine learning model by splitting the dataset into k subsets (folds).
- The model is trained k times, each time using $k-1$ folds for training and the remaining one for testing.
- The final model performance is the average of all k iterations.



Steps in k-Fold Cross-Validation

- Shuffle the dataset (optional but recommended).
- Split the dataset into k equal-sized subsets (folds).
- Train the model k times, each time using k-1 folds for training and the remaining fold for validation.
- Compute the evaluation metric (e.g., accuracy, F1-score) for each fold.
- Average the results from all folds to get the final model performance.

10-Fold Cross Validation



Let's say we have **10 data samples**, labeled S1 to S10:

Sample

S1

S2

S3

S4

S5

S6

S7

S8

S9

S10

We want to use **5-Fold Cross-Validation**, so:

- Total samples = 10
- Number of folds (**k**) = 5
- Each fold will contain 2 samples ($10 \div 5$)

Let's divide the dataset into 5 folds:

- **Fold 1:** [S1, S2]
- **Fold 2:** [S3, S4]
- **Fold 3:** [S5, S6]
- **Fold 4:** [S7, S8]
- **Fold 5:** [S9, S10]



Iteration 1

Test: Fold 1 → [S1, S2]

Train: [S3, S4, S5, S6, S7, S8, S9, S10]



Iteration 2

Test: Fold 2 → [S3, S4]

Train: [S1, S2, S5, S6, S7, S8, S9, S10]



Iteration 3

Test: Fold 3 → [S5, S6]

Train: [S1, S2, S3, S4, S7, S8, S9, S10]



Iteration 4

Test: Fold 4 → [S7, S8]

Train: [S1, S2, S3, S4, S5, S6, S9, S10]



Iteration 5

Test: Fold 5 → [S9, S10]

Train: [S1, S2, S3, S4, S5, S6, S7, S8]

Fold

Accuracy

1

80%

2

90%

3

70%

4

85%

5

75%

Average Accuracy = $(80 + 90 + 70 + 85 + 75) \div 5 = \mathbf{80\%}$

Small dataset with 10 samples and 2 features

Index	Feature_1	Feature_2	Target
0	2.5	1.3	0
1	3.1	2.0	1
2	1.2	0.7	0
3	4.5	3.2	1
4	3.3	1.8	1
5	5.1	2.9	1
6	2.8	1.5	0
7	4.0	2.2	1
8	1.5	0.9	0
9	3.7	2.4	1

We split the dataset into 5 folds.

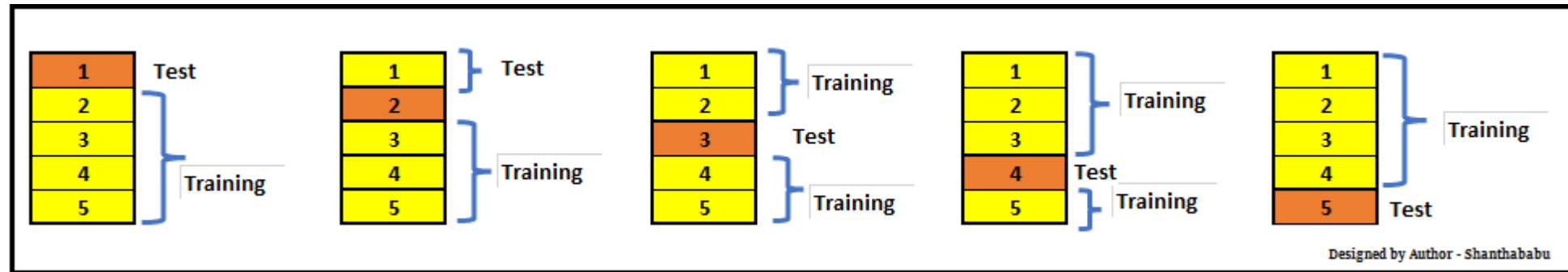
Fold Splits (after shuffling):

Fold	Train Indices	Test Index
1	[2, 8, 6, 1, 5, 7, 9, 4, 3]	[0]
2	[0, 8, 6, 1, 5, 7, 9, 4, 3]	[2]
3	[0, 2, 8, 1, 5, 7, 9, 4, 3]	[6]
4	[0, 2, 8, 6, 5, 7, 9, 4, 3]	[1]
5	[0, 2, 8, 6, 1, 7, 9, 4, 3]	[5]

The accuracy varies across 5 folds.

- 4 out of 5 folds had 100% accuracy.
- 1 fold (Fold 3) had 0% accuracy due to a misclassification.
- **The final average accuracy is 80% (0.8000).**

Let's have a generalized K value. If $K=5$, it means, in the given dataset and we are splitting into 5 folds and running the Train and Test. During each run, one fold is considered for testing and the rest will be for training and moving on with iterations, the below pictorial representation would give you an idea of the flow of the fold-defined size.



The Problem with Imbalanced Data

- Standard K-Fold cross-validation, while a robust method for many scenarios, randomly divides the dataset into K equal-sized folds. This randomness does not guarantee the representation of minority classes in each fold.
- This could severely impact the model's ability to learn and generalize about those minority classes, leading to biased or inaccurate models.

Stratified k-Fold Cross-Validation

- Stratified k-Fold Cross-Validation is a variation of k-Fold CV that ensures each fold maintains the **same proportion of class labels** as the entire dataset.
- This is particularly useful for **imbalanced datasets**, where some classes appear much less frequently than others.
- Prevents **biased training** when class distributions are uneven.
- Ensures that **each fold** has a similar **class proportion** as the full dataset.
- Useful for **classification problems**, especially when some classes are rare.

Order Samples Per Class: First, the samples in the dataset are ordered by class, grouping all samples belonging to the same class together.

Creating Strata: For each class, the samples are divided into K non-overlapping sets or strata. It ensures each class is proportionally represented in every fold. These strata are created in such a way that the number of samples in each stratum is approximately equal, or as close as possible if the class size is not divisible by K .

Combining Strata into Folds: The stratified folds are then created by taking the first stratum from each class and combining them into the first fold, the second stratum from each class into the second fold, and so on. This way, we form folds that reflect the dataset's original class distribution.

Perform K-Fold Cross-Validation: The K-Fold Cross-Validation process is then carried out as usual, with one fold serving as the test set and the remaining $K-1$ folds used for training in each iteration.

10-sample dataset with 2 features and an imbalanced class distribution.

Class Distribution: Class 0: 4 samples Class 1: 6 samples

Sample (X1, X2)	Target (y)
(2.5, 1.3)	0
(3.1, 2.0)	1
(1.2, 0.7)	0
(4.5, 3.2)	1
(3.3, 1.8)	1
(5.1, 2.9)	1
(2.8, 1.5)	0
(4.0, 2.2)	1
(1.5, 0.9)	0
(3.7, 2.4)	1

Stratified k-Fold Cross-Validation to maintain class proportions across 5 folds.
Dataset into 5 folds, ensuring that each fold has a similar class proportion.

Fold-wise Splitting and Training

Fold	Train Indices	Test Indices	Train Class Distribution	Test Class Distribution
1	[0,1,2,3,4,7,8,9]	[5,6]	[3 Class 0, 5 Class 1]	[1 Class 0, 1 Class 1]
2	[1,2,4,5,6,7,8,9]	[0,3]	[3 Class 0, 5 Class 1]	[1 Class 0, 1 Class 1]
3	[0,1,2,3,4,5,6,7]	[8,9]	[3 Class 0, 5 Class 1]	[1 Class 0, 1 Class 1]
4	[0,3,4,5,6,7,8,9]	[1,2]	[3 Class 0, 5 Class 1]	[1 Class 0, 1 Class 1]
5	[0,1,2,3,5,6,8,9]	[4,7]	[4 Class 0, 4 Class 1]	[0 Class 0, 2 Class 1]

Predictions and Accuracies

Fold	Actual Labels	Predicted Labels	Accuracy
1	[1, 0]	[1, 0]	1.0 (100%)
2	[0, 1]	[0, 1]	1.0 (100%)
3	[0, 1]	[0, 1]	1.0 (100%)
4	[1, 0]	[1, 0]	1.0 (100%)
5	[1, 1]	[1, 1]	1.0 (100%)

✔ The model predicts perfectly in all folds.

Measuring classifier performance

- Evaluating a machine learning classifier involves several key metrics. These help us understand how well the model is making predictions.

Common Classification Metrics

- Accuracy
- Precision (Positive Predictive Value)
- Recall (Sensitivity or True Positive Rate)
- F1 Score (Harmonic Mean of Precision & Recall)
- Confusion Matrix

Accuracy

Accuracy is the proportion of correctly classified instances out of the total instances. It is one of the most commonly used metrics to measure a classifier's performance.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- **TP (True Positives):** Correctly predicted positive cases
- **TN (True Negatives):** Correctly predicted negative cases
- **FP (False Positives):** Incorrectly predicted as positive
- **FN (False Negatives):** Incorrectly predicted as negative

Good for balanced datasets, but misleading for imbalanced data.

Accuracy :Example

Imagine a classifier that predicts whether an email is spam or not. If we have the following results:

- **TP = 50** (Spam emails correctly classified as spam)
- **TN = 40** (Non-spam emails correctly classified as non-spam)
- **FP = 5** (Non-spam emails wrongly classified as spam)
- **FN = 5** (Spam emails wrongly classified as non-spam)

$$\text{Accuracy} = \frac{50 + 40}{50 + 40 + 5 + 5} = \frac{90}{100} = 90\%$$

Limitations of Accuracy

- Accuracy is useful when the dataset is **balanced** (i.e., classes have nearly equal representation). However, it can be misleading when the dataset is **imbalanced** (one class significantly outnumbers the other).
- **Example of misleading accuracy:**
If a dataset contains **95% negative** cases and **5% positive** cases, a model that always predicts "negative" will have **95% accuracy** but is useless for detecting positive cases.
- To handle imbalanced datasets, consider additional metrics like **Precision, Recall, F1-Score, and MCC**.

Precision (Positive Predictive Value)

Precision (also called Positive Predictive Value) measures how many of the predicted positive instances are actually correct. It is particularly useful when false positives (FP) are costly, such as in spam detection.

$$Precision = \frac{TP}{TP + FP}$$

Where:

TP (True Positives): Correctly predicted positive instances

FP (False Positives): Incorrectly predicted positive instances (Type I error)

Punishes Wrong Alarms

Precision: Example

- Imagine a classifier predicting whether an email is **spam** (positive class) or **not spam** (negative class).
- If the model predicts 100 emails as spam, but only **80** are actually spam, and **20** are wrongly classified as spam:
 - **TP = 80** (Correctly predicted spam emails)
 - **FP = 20** (Non-spam emails wrongly classified as spam)

$$\text{Precision} = \frac{80}{80 + 20} = \frac{80}{100} = 0.8 \quad (80\%)$$

When is Precision Important?

- When false positives are more harmful than false negatives.
- Example scenarios:
 - **Spam filtering:** Predicting an important email as spam (FP) could cause a user to miss critical messages.

Recall (Sensitivity or True Positive Rate)

Recall (also called Sensitivity or True Positive Rate) measures how many of the actual positive instances were correctly predicted by the model. It is crucial when missing positive cases (**false negatives**) is costly, such as in medical diagnoses

$$\text{Recall} = \frac{TP}{TP + FN}$$

Where:

- **TP (True Positives)**: Correctly predicted positive instances
- **FN (False Negatives)**: Positive instances incorrectly classified as negative (Type II error)

Punishes Missed Cases

Recall : Example

- Imagine a medical test for **cancer detection**:
 - **100 patients have cancer (actual positives)**.
 - The model correctly identifies **80** as cancerous (**TP = 80**).
 - However, it misses **20** cases (**FN = 20**).

$$\text{Recall} = \frac{80}{80 + 20} = \frac{80}{100} = 0.8 \quad (80\%)$$

This means the model correctly detects 80% of cancer cases, but misses 20%.

When is Recall Important?

- When false negatives (FN) are more **dangerous** than false positives (FP).
- Example scenarios:
 - **Medical Diagnosis:** Missing a cancer diagnosis (FN) is much worse than a false alarm (FP).
 - **Security Systems:** Letting an intruder pass undetected (FN) is more critical than a false alert (FP).

F1 Score

The F1-score is the harmonic mean of Precision and Recall. It provides a single metric that balances both values, making it useful when a dataset has an imbalanced class distribution (i.e., one class is much more frequent than the other)

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

F1 Score

Fraud Detection (Balanced/Context Dependent)

Scenario: A bank flags credit card transactions.

High Recall (Focus): The system flags every transaction that looks slightly suspicious, reducing the chance of missed fraud, even if many valid customer transactions are temporarily blocked.

High Precision (Focus): The system only flags transactions it is absolutely certain are fraudulent. This results in fewer customer complaints about blocked cards, but actual fraudulent transactions might go unnoticed

Example Calculation:

Imagine a model predicting whether an email is spam:

- True Positives (TP) = 80 (Correctly classified spam emails)
- False Positives (FP) = 20 (Non-spam emails wrongly classified as spam)
- False Negatives (FN) = 30 (Spam emails wrongly classified as non-spam)

First, calculate Precision and Recall:

$$\text{Precision} = \frac{80}{80 + 20} = \frac{80}{100} = 0.8$$

$$\text{Recall} = \frac{80}{80 + 30} = \frac{80}{110} \approx 0.727$$

Now, compute the F1-score:

$$F1 = 2 \times \frac{0.8 \times 0.727}{0.8 + 0.727}$$

$$F1 = 2 \times \frac{0.5816}{1.527} \approx 0.761$$

Thus, the F1-score is 0.761 (76.1%).

When is F1-Score Important?

- F1-score is useful when:
 - The dataset is **imbalanced** (e.g., fraud detection, rare disease classification).
 - Both **false positives (FP)** and **false negatives (FN)** need to be minimized.
 - A model must balance **precision and recall**, instead of focusing on just one.

Exercise Problem

- A model for disease detection reports:
- Accuracy = 97%
- Recall = 0.22

Questions:

What does this tell you about the dataset?

What does Recall = 0.22 imply in real life?

Would you approve this model for hospital deployment?

Answer

- **What does this tell you about the dataset?**

This strongly suggests the dataset is highly imbalanced.

Why?

- If Recall is only 0.22, the model is detecting only 22% of actual disease cases.
- Yet Accuracy is 97%, meaning overall predictions are mostly correct
- This can only happen if the majority of patients are healthy.

So most likely:

- Disease cases are rare.
 - The model is correctly predicting healthy cases
 - It is missing many actual disease cases.
- In other words:
 - The model is performing well on the majority class and poorly on the minority (disease) class.

What does Recall = 0.22 imply in real life?

Recall = 0.22 means:

- Out of 100 actual sick patients, the model correctly detects only 22.

That means:

- 78% of sick patients are missed (False Negatives).

In real life:

- Patients may go untreated. Disease may worsen.
- Legal and ethical consequences may occur.
- The hospital may lose trust.
- In healthcare, False Negatives are often far more dangerous than False Positives.

Would you approve this model for hospital deployment?

- No.

Reason:

- Missing 78% of actual patients is unacceptable in medical diagnosis.
- High accuracy is misleading.
- The model is effectively acting like a “mostly healthy predictor.”
- This model is unsafe for deployment.

Confusion Matrix

- A **confusion matrix** is a table that helps evaluate a classification model's performance by **showing the actual vs. predicted classifications**. It is especially useful for understanding model errors.
- A table that shows the counts of **TP, TN, FP, FN** for a classifier. Example:

Confusion Matrix

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Example Calculation:

Imagine a classifier predicting whether an email is **spam** (positive class) or **not spam** (negative class).

After testing, we get the following results:

Actual / Predicted	Spam (Positive)	Not Spam (Negative)
Spam Emails (Actual Positive)	50 (TP)	10 (FN)
Not Spam Emails (Actual Negative)	5 (FP)	35 (TN)

Here's how we interpret this:

- ✅ 50 emails were correctly classified as spam (TP).
- ❌ 10 spam emails were missed (FN).
- ❌ 5 non-spam emails were wrongly flagged as spam (FP).
- ✅ 35 emails were correctly classified as non-spam (TN).

n=165	Predicted: NO	Predicted: YES
Actual: NO	50	10
Actual: YES	5	100

What can we learn from this matrix?

- There are two possible predicted classes: "yes" and "no". If we were predicting the presence of a disease, for example, "yes" would mean they have the disease, and "no" would mean they don't have the disease.
- The classifier made a total of 165 predictions (e.g., 165 patients were being tested for the presence of that disease).
- Out of those 165 cases, the classifier predicted "yes" 110 times, and "no" 55 times.
- In reality, 105 patients in the sample have the disease, and 60 patients do not.

n=165		Predicted: NO	Predicted: YES	
Actual: NO		TN = 50	FP = 10	60
Actual: YES		FN = 5	TP = 100	105
		55	110	

- **Accuracy:** Overall, how often is the classifier correct?
 - $(TP+TN)/total = (100+50)/165 = 0.91$
- **Misclassification Rate:** Overall, how often is it wrong?
 - $(FP+FN)/total = (10+5)/165 = 0.09$
 - equivalent to 1 minus Accuracy
 - also known as "Error Rate"

n=165	Predicted: NO	Predicted: YES	
Actual: NO	TN = 50	FP = 10	60
Actual: YES	FN = 5	TP = 100	105
	55	110	

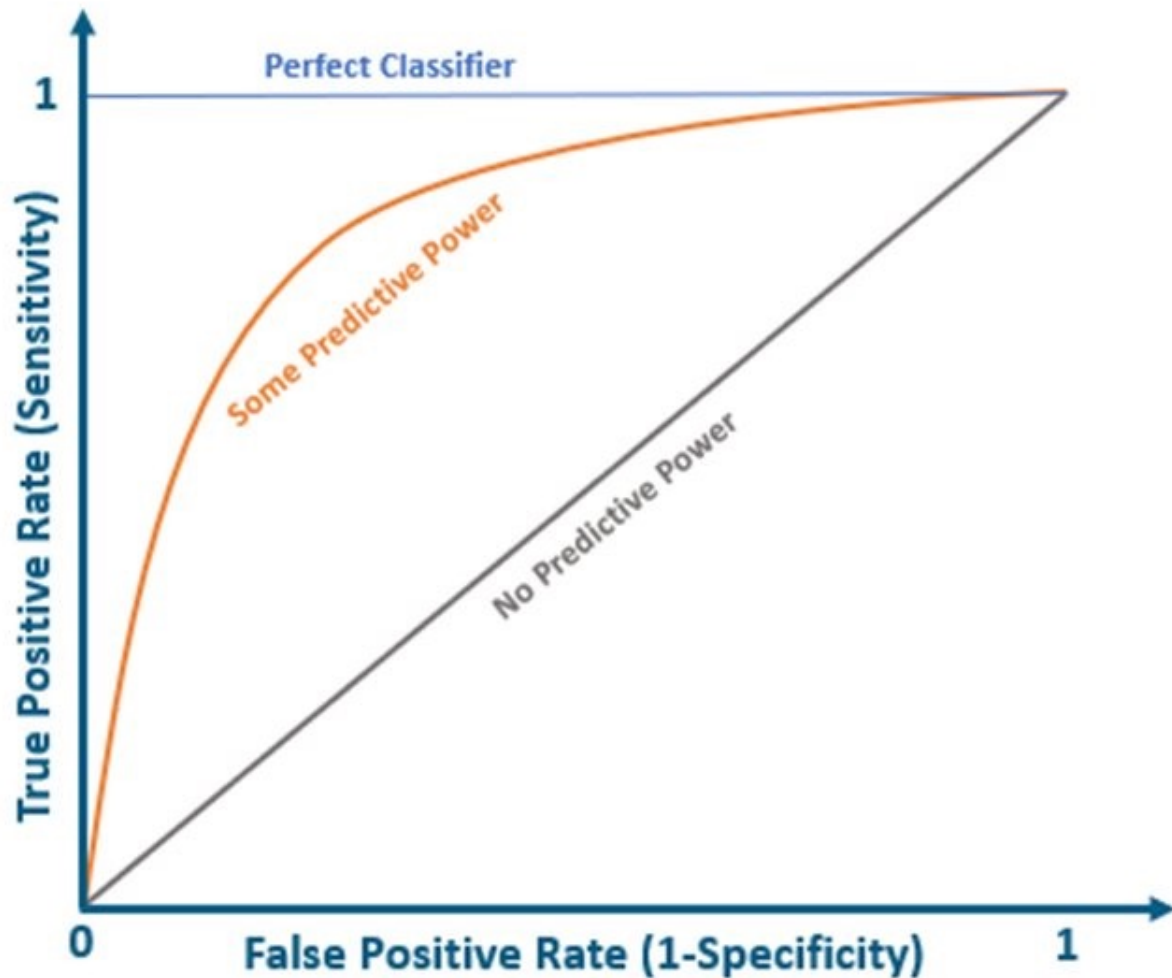
- **True Positive Rate:** When it's actually yes, how often does it predict yes?
 - $TP/\text{actual yes} = 100/105 = 0.95$
 - also known as "**Sensitivity**" or "**Recall**"
- **False Positive Rate:** When it's actually no, how often does it predict yes?
 - $FP/\text{actual no} = 10/60 = 0.17$
- **True Negative Rate:** When it's actually no, how often does it predict no?
 - $TN/\text{actual no} = 50/60 = 0.83$
 - equivalent to 1 minus False Positive Rate
 - also known as "**Specificity**"

n=165	Predicted: NO	Predicted: YES	
Actual: NO	TN = 50	FP = 10	60
Actual: YES	FN = 5	TP = 100	105
	55	110	

- **Precision:** When it predicts yes, how often is it correct?
 - $TP / \text{predicted yes} = 100 / 110 = 0.91$
- **Prevalence:** How often does the yes condition actually occur in our sample?
 - $\text{actual yes} / \text{total} = 105 / 165 = 0.64$

ROC Curve

- The **receiver operating characteristic (ROC) curve** is frequently used for evaluating the performance of binary [Classification](#) algorithms
- ROC is a probability curve that depicts the TPR (rate of true positives) on the y-axis against the FPR (rate of false positives) on the x-axis.
- It essentially distinguishes the 'signal' from the 'noise', thereby highlighting the Sensitivity of the classifier model.
- Measures how well the model differentiates between classes.



- The graph below demonstrates theoretically how the ROC curve would be plotted for a classifier model.
- A perfect classifier will have a ROC where the graph would hit a true positive rate of 100% with zero false positives.
- The grey diagonal line represents a classifier with no predictive power – one that guesses randomly

- Nearly all real-world classifiers would fall between these two extremes – having some predictive power. Generally, we look for a classifier that maintains a high TPR while also low FPR.
- ROC Curves help determine the exact trade-off between the TPR and FPR for a classifier using different measures of probability thresholds.

Which Model should you use?

	False Positive Rate	False Negative Rate
Model 1	41%	3%
Model 2	5%	25%

Actually, the same model
- different thresholds

Mistakes have different costs:

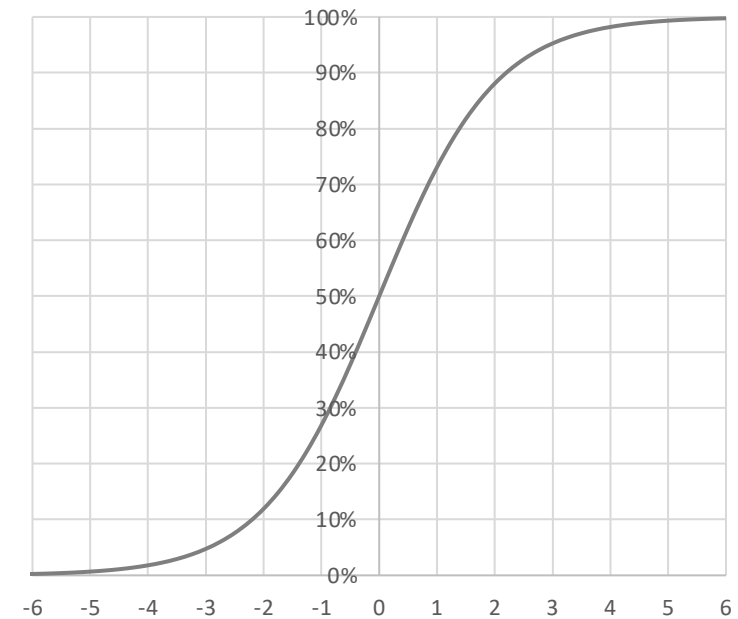
- Disease Screening – LOW FN Rate
- Spam filtering – LOW FP Rate

Conservative vs Aggressive settings:

- The same application might need multiple tradeoffs

Classifications and Probability Estimates

- Logistic regression produces a score between 0 – 1 (probability estimate)
- Use threshold to produce classification
- What happens if you vary the threshold?



Example of Changing Thresholds

Score	Prediction	Y
.25		0
.45		0
.55		1
.67		0
.82		1
.95		1

	Threshold = .5
False Positive Rate	
False Negative Rate	

	Threshold = .6
False Positive Rate	
False Negative Rate	

	Threshold = .7
False Positive Rate	
False Negative Rate	

ROC Curve (Receiver Operating Characteristic)

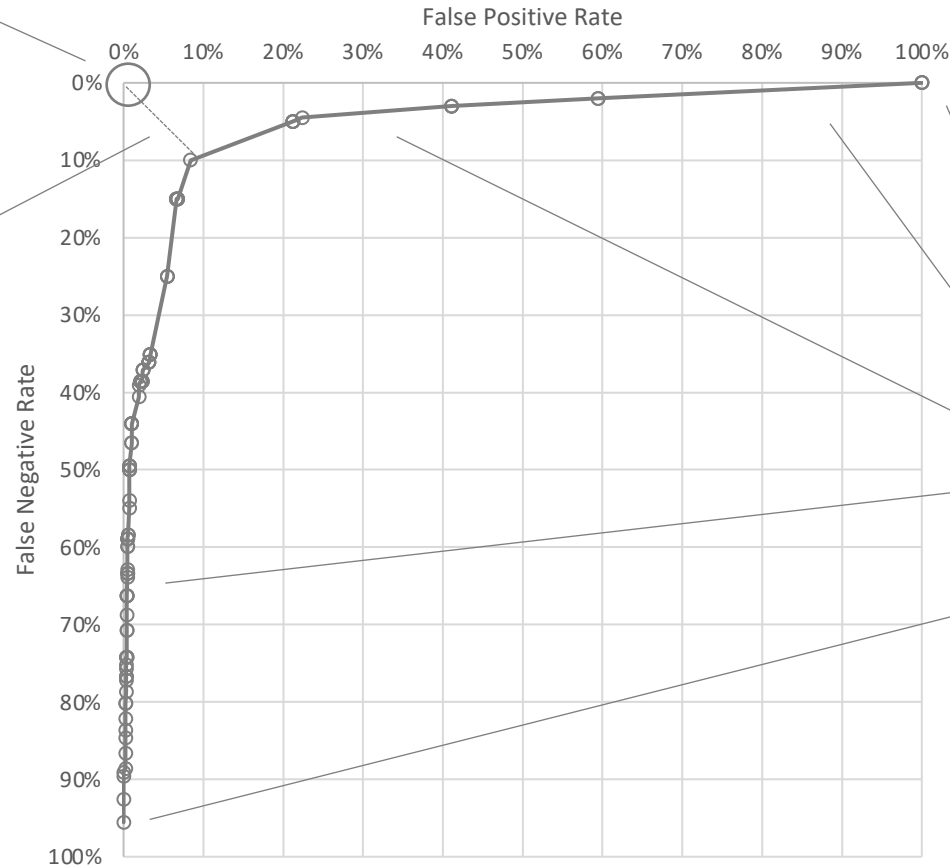
Perfect score:

- 0% of 1s called 0
- 0% of 0s called 1

Percent of 0s classified as 1

This model's distance from perfect

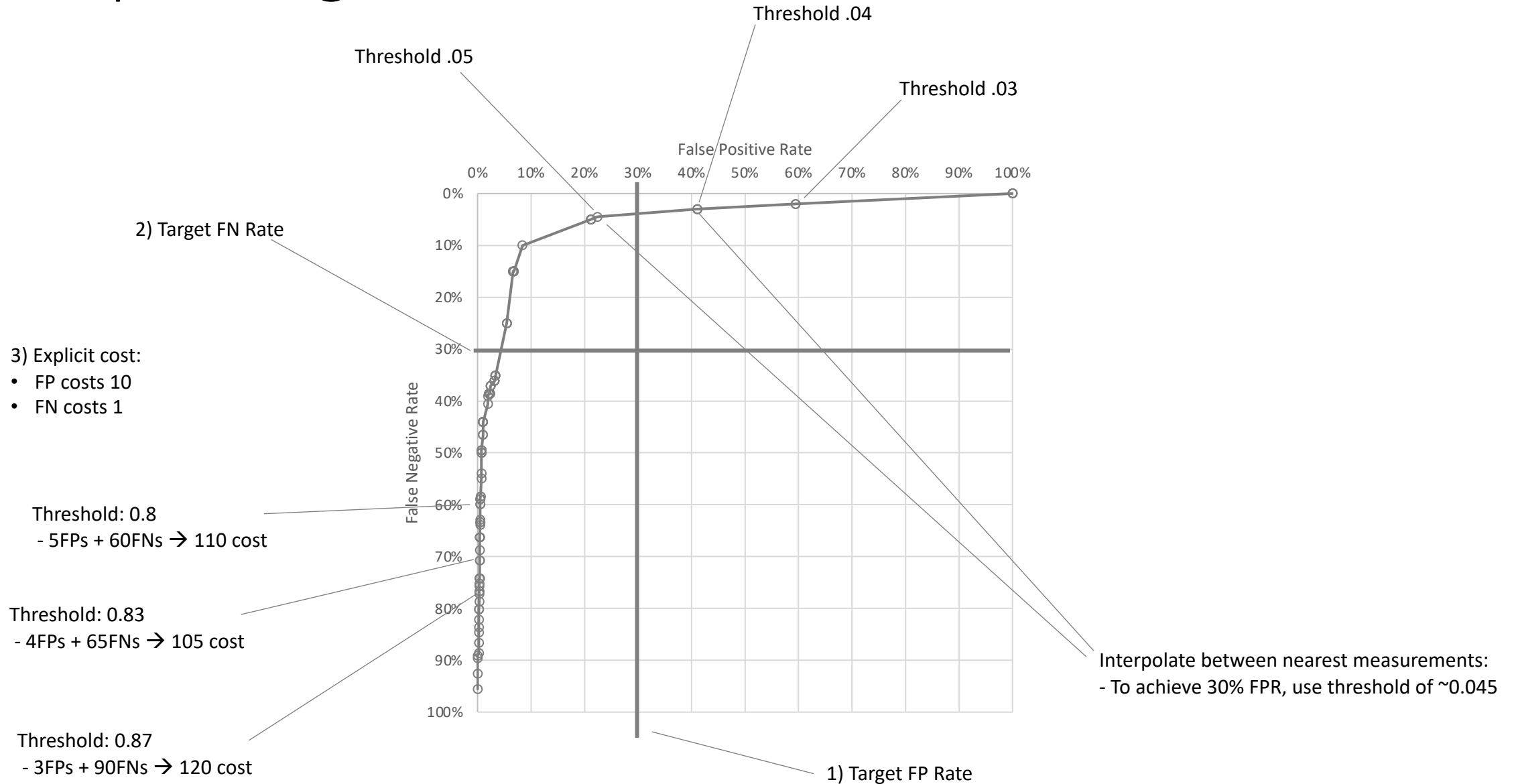
Percent of 1s classified as 0



Sweep threshold from 0 to 1

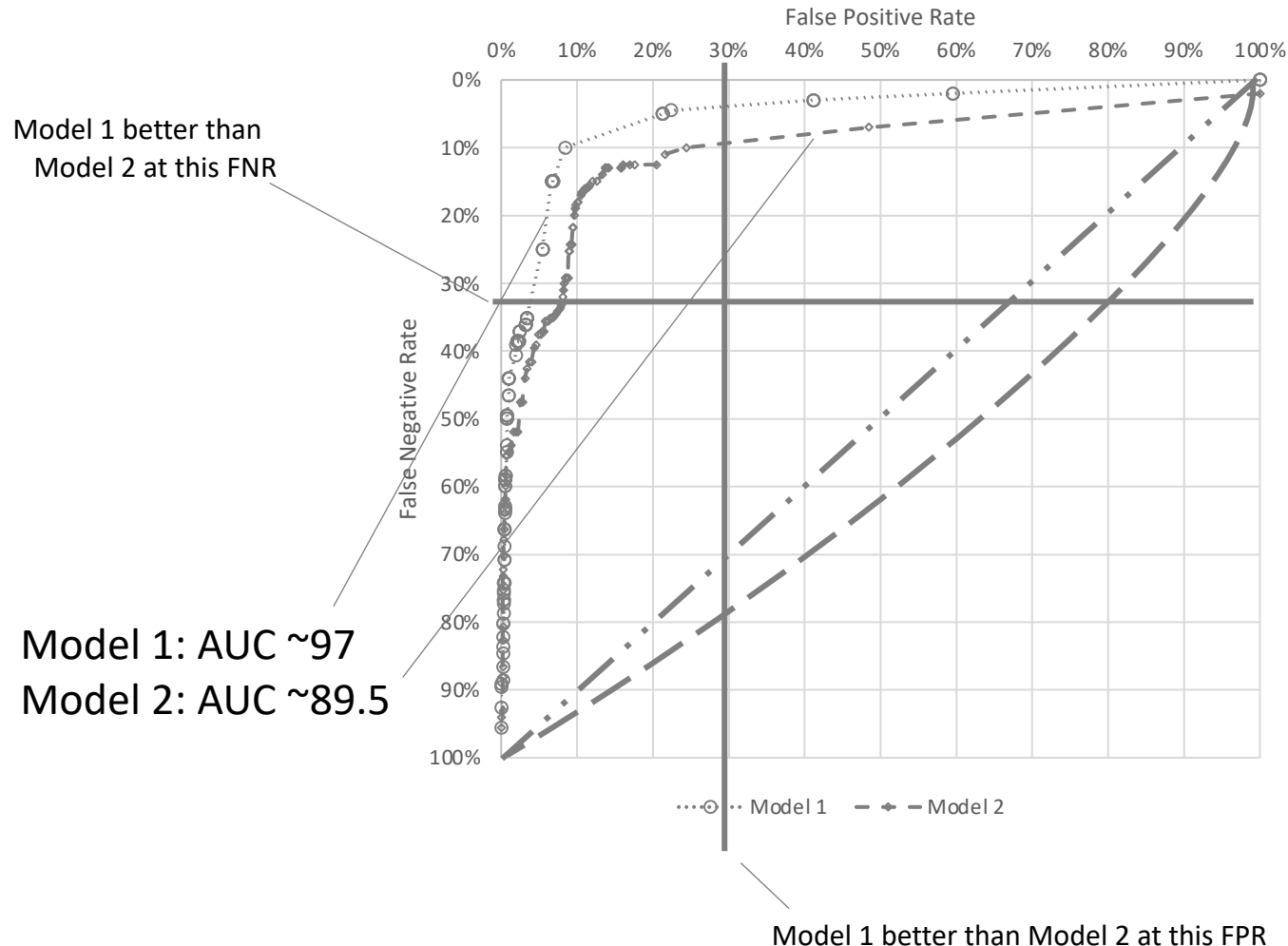
- Threshold 0: 'all' classified as 1
- Threshold 1: 'all' classified as 0

Operating Points



Comparing Models with ROC Curves

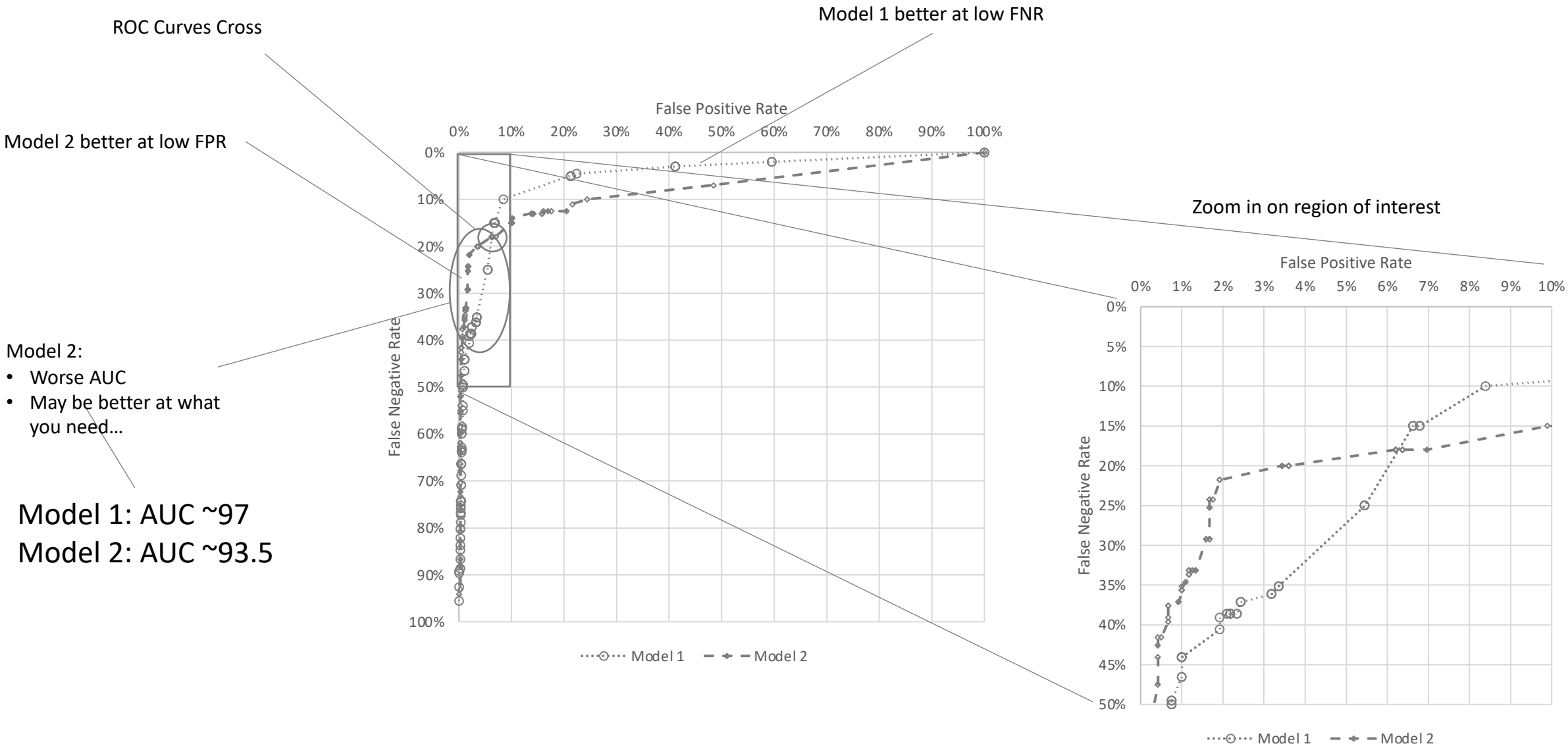
Model 1 better than Model 2 at every FPR or FNR target



Area Under Curve (AUC)

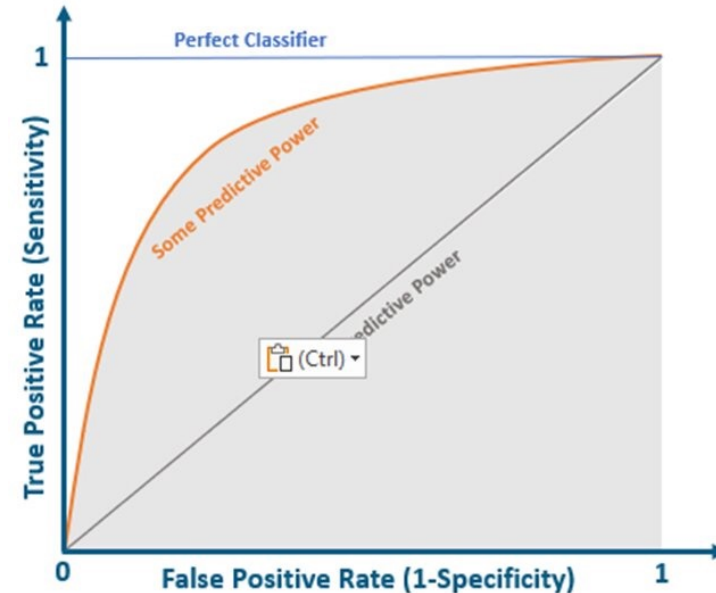
- Integrate Area under the curve
- Perfect score is 1
- Higher scores allow for generally better tradeoffs
- AUC of 0.5 indicates model is essentially randomly guessing
- AUC of < 0.5 indicates you're doing something wrong...

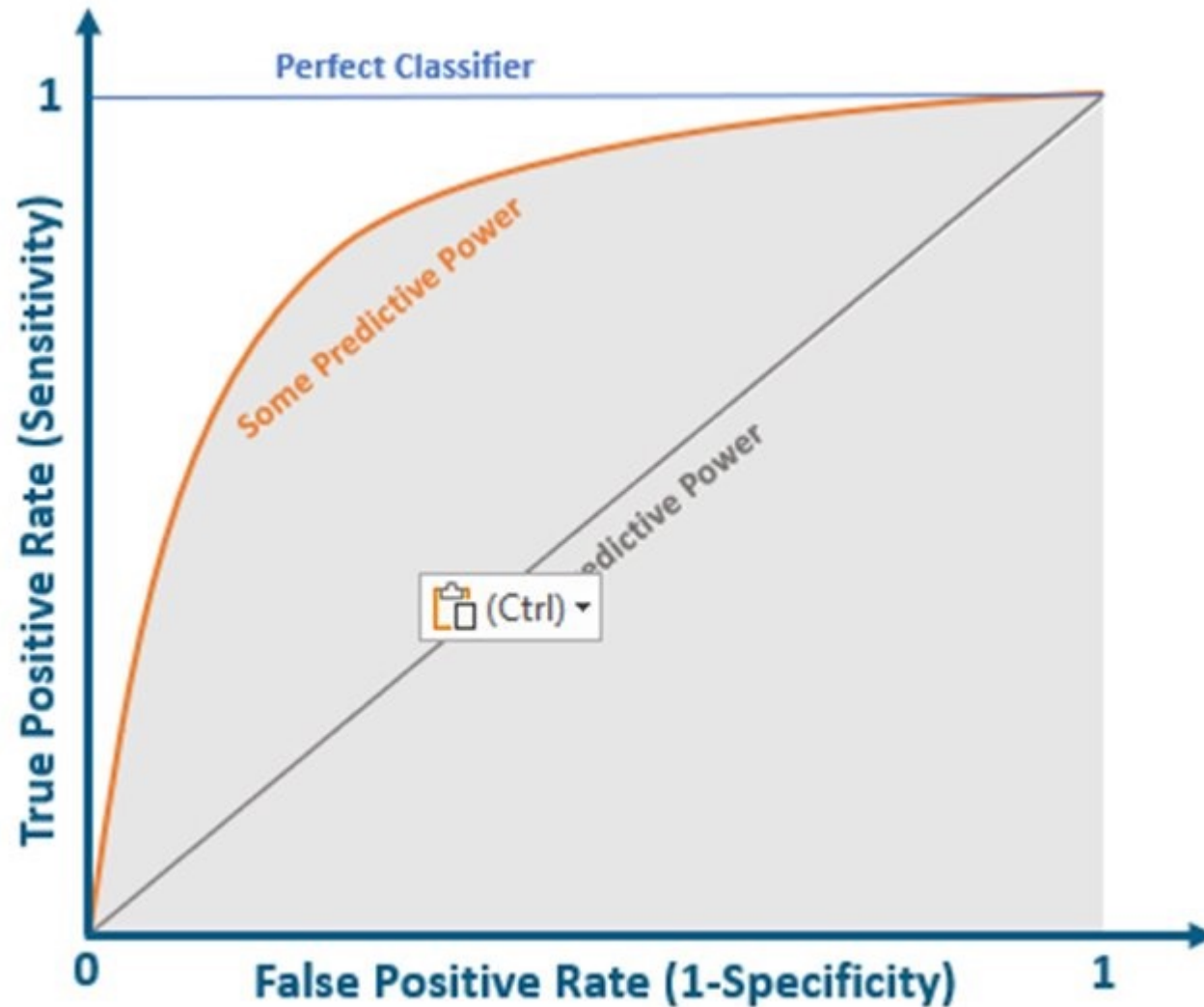
More ROC Comparisons



Area Under the Curve (AUC)

- It is another important evaluation metric generally used for binary classification problems. AUC represents the degree of separability. The higher the AUC, the better the classifier can distinguish between the positive and negative classes.
- The graph below demonstrates the area under the curve. AUC is used to summarize the ROC Curve as it measures the entire 2D area present underneath the ROC curve.





The value of AUC ranges from 0 to 1. If $AUC=1$, the model can distinguish between the Positive and the Negative class points correctly.

Similarly, a model with 100% false predictions, i.e., predicting all Negatives as Positives, and all Positives as Negatives, would have $AUC=0$.

When an $AUC=0.5$, it means the model possesses no class separation capacity and cannot distinguish between the Positive and Negative class points.

Difference Between ROC and AUC

Feature	ROC (Receiver Operating Characteristic) Curve	AUC (Area Under the Curve)
Definition	A graphical plot showing the trade-off between True Positive Rate (Recall) and False Positive Rate at different thresholds.	A numerical value representing the total area under the ROC curve, summarizing the classifier's performance.
Purpose	Visualizes the performance of a classification model across various thresholds.	Quantifies the model's ability to distinguish between positive and negative classes.
Interpretation	Helps understand the trade-off between sensitivity and specificity.	A higher AUC means a better-performing model.
Range	The ROC curve extends from (0,0) to (1,1) on a graph.	AUC value ranges from 0 to 1 (with 1 being perfect and 0.5 being random guessing).
Use Case	Used to analyze and compare different classification models.	Used to summarize the performance of a classifier in a single metric.

Comparing Two Classification Models Using a Paired t-Test

A paired t-test is used to determine whether the performance difference between two classification models evaluated on the same dataset is statistically significant.

Why We Use a Paired t-Test

- When comparing models like:
 - Logistic Regression
 - Random Forest
- their accuracies may differ slightly.

Split	Model A Accuracy	Model B Accuracy
1	0.84	0.86
2	0.82	0.85
3	0.83	0.84
4	0.85	0.87
5	0.81	0.84

Is Model B really better, or is the difference due to random sampling?

The **paired t-test** answers this.

Paired t-Test

Formula for the Paired t-Test

$$t = \frac{\bar{D}}{s_D / \sqrt{n}}$$

where:

- $D_i = X_i - Y_i$ is the difference in performance for each test set.
- $\bar{D}$ = Mean of the differences.
- s_D = Standard deviation of the differences.
- n = Number of test sets (or trials).

t-value

- The **t-value** measures **how far the observed result is from the null hypothesis**.

For paired t-test:

$$t = \frac{\bar{d}}{s_d / \sqrt{n}}$$

Where

- $\bar{d}$ = mean difference
- s_d = standard deviation of differences
- n = number of samples

A large **|t|** value means the observed difference is far from 0.

Null Hypothesis

Suppose we compare **Model A** and **Model B**.

Null hypothesis:

$$H_0 : \mu_A = \mu_B$$

Meaning:

Both models have the same performance.

If statistical testing shows strong evidence against this assumption, we **reject the null hypothesis**.

Steps for Comparing Two Models Using a Paired t-Test

Step 1: Train and Evaluate Both Models

- Choose two classification models (e.g., **Random Forest vs. SVM**).
- Train and test them on **the same dataset** multiple times (e.g., **5 different train-test splits**).
- Record their performance scores (e.g., accuracy).

Step 2: Compute Performance Differences

- Calculate the performance difference for each test set:

$$D_i = X_i - Y_i$$

where:

- X_i = Accuracy of Model A on test set i .
- Y_i = Accuracy of Model B on test set i .

Step 3: Perform the Paired t-Test

- Compute:
 - Mean of differences $\bar{D}$.
 - Standard deviation of differences s_D .
 - t-statistic using the formula.
- Compare with the critical t-value or compute the p-value.

Numerical Example

- Let's compare **Random Forest (Model A)** and **SVM (Model B)** on 5 different train-test splits.

Test Split	Model A Accuracy (%)	Model B Accuracy (%)	Difference (A - B)
1	85	83	2
2	87	85	2
3	90	88	2
4	86	87	-1
5	88	86	2

Step 1: Compute the Mean Difference

$$\bar{D} = \frac{(2 + 2 + 2 - 1 + 2)}{5} = \frac{7}{5} = 1.4$$

Step 2: Compute the Standard Deviation of the Differences

First, compute squared deviations:

$$(2 - 1.4)^2 = 0.36$$

$$(2 - 1.4)^2 = 0.36$$

$$(2 - 1.4)^2 = 0.36$$

$$(-1 - 1.4)^2 = (-2.4)^2 = 5.76$$

$$(2 - 1.4)^2 = 0.36$$

Standard Deviation of Differences

$$s_d = \sqrt{\frac{\sum (d_i - \bar{d})^2}{n - 1}}$$

Sum of squared differences:

$$0.36 + 0.36 + 0.36 + 5.76 + 0.36 = 7.2$$

Now, compute the standard deviation:

$$s_D = \sqrt{\frac{7.2}{5 - 1}} = \sqrt{\frac{7.2}{4}} = \sqrt{1.8} \approx 1.34$$

Step 3: Compute the t-Statistic

$$t = \frac{\bar{D}}{s_D / \sqrt{n}}$$

$$t = \frac{1.4}{1.34 / \sqrt{5}} = \frac{1.4}{0.6} = 2.33$$

Step 4: Determine the p-Value

- For **df = 5 - 1 = 4** at a **95% confidence level**, the critical t-value for a two-tailed test is **2.776**.
- If **$|t| \geq 2.776$** , reject H_0 (significant difference).
- If **$|t| < 2.776$** , fail to reject H_0 (no significant difference).
- Since **$2.33 < 2.776$** , we **fail to reject the null hypothesis**, meaning **no significant difference** between the models at a 95% confidence level.

When to Use This Method

Use a paired t-test when:

- You compare **two models using multiple independent train-test splits**.
- You want to determine if one model significantly outperforms the other.
- You are using **hold-out validation** instead of **cross-validation**.

Don't use it when:

- You're using **K-fold cross-validation** → Use a **K-fold CV paired t-test** instead.
- You have **only one train-test split** → Not enough data for statistical testing.

Exercise problem

- Two machine learning classifiers are evaluated on the **same 6 datasets**.
- Classifier A → Random Forest
- Classifier B → Support Vector Machine (SVM)
- The accuracy obtained on each dataset is given below.

Dataset	Classifier A	Classifier B
D1	0.88	0.84
D2	0.90	0.86
D3	0.87	0.85
D4	0.91	0.88
D5	0.89	0.87
D6	0.92	0.90

Answer

- $t=7.08 > 2.571$
- Reject the null hypothesis
- There is **significant evidence that Classifier A performs better than Classifier B.**

Comparing Two Classification Algorithms Using K-Fold Cross-Validation (K-Fold CV) Paired t-Test

- When comparing two classification models, **K-Fold Cross-Validation (CV) paired t-test** is a robust statistical method.
- It reduces variance by evaluating both models on the **same K-fold splits**, making the comparison more reliable than a single train-test split.

K-Fold CV Paired t-Test

- A **paired t-test** checks whether the difference in performance between two models is statistically significant.
- In **K-Fold CV paired t-test**, both models are evaluated on **K different folds**, and a paired t-test is applied to the K-fold results.

Formula for the Paired t-Test

$$t = \frac{\bar{D}}{s_D / \sqrt{K}}$$

where:

- $D_i = X_i - Y_i$ is the difference in performance (e.g., accuracy) for each fold.
- $\bar{D}$ = Mean of the differences across K folds.
- s_D = Standard deviation of the differences.
- K = Number of folds.

Step 1: Perform K-Fold Cross-Validation

- Choose two classification models (e.g., Random Forest vs. SVM).
- Perform K-fold cross-validation (e.g., $K = 10$).
- Compute the performance metric (e.g., accuracy) for both models on each fold.

Step 2: Compute Performance Differences

- Calculate $D_i = X_i - Y_i$ for each fold.

Step 3: Conduct the Paired t-Test

- Compute:
 - Mean of the differences $\bar{D}$.
 - Standard deviation of the differences s_D .
 - t-statistic using the formula.
- Compare with the critical t-value or compute the p-value.

Let's compare **Random Forest (Model A)** and **SVM (Model B)** using **10-fold cross-validation**.

Fold	Model A Accuracy (%)	Model B Accuracy (%)	Difference (A - B)
1	85	83	2
2	87	85	2
3	90	88	2
4	86	87	-1
5	88	86	2
6	89	87	2
7	85	84	1
8	86	85	1
9	90	88	2
10	88	86	2

Step 1: Compute the Mean Difference

$$\bar{D} = \frac{(2 + 2 + 2 - 1 + 2 + 2 + 1 + 1 + 2 + 2)}{10} = \frac{15}{10} = 1.5$$

Step 2: Compute the Standard Deviation of the Differences

1. Compute squared deviations:

$$(2 - 1.5)^2 = 0.25, \quad (2 - 1.5)^2 = 0.25, \quad (2 - 1.5)^2 = 0.25, \quad (-1 - 1.5)^2 = 6.25, \quad (2 - 1.5)^2 = 0.25$$

$$(2 - 1.5)^2 = 0.25, \quad (1 - 1.5)^2 = 0.25, \quad (1 - 1.5)^2 = 0.25, \quad (2 - 1.5)^2 = 0.25, \quad (2 - 1.5)^2 = 0.25$$

2. Sum of squared differences:

$$0.25 + 0.25 + 0.25 + 6.25 + 0.25 + 0.25 + 0.25 + 0.25 + 0.25 + 0.25 = 8.5$$

3. Compute standard deviation:

$$s_D = \sqrt{\frac{8.5}{10 - 1}} = \sqrt{\frac{8.5}{9}} = \sqrt{0.94} \approx 0.97$$

Step 3: Compute the t-Statistic

$$t = \frac{\bar{D}}{s_D / \sqrt{K}}$$
$$t = \frac{1.5}{0.97 / \sqrt{10}} = \frac{1.5}{0.31} = 4.83$$

Step 4: Determine the p-Value

- For **$df = 10 - 1 = 9$** at a **95% confidence level**, the critical t-value for a **two-tailed test** is **2.262**.
- If **$|t| \geq 2.262$** , reject H_0 (significant difference).
- If **$|t| < 2.262$** , fail to reject H_0 (no significant difference).
- Since **$4.83 > 2.262$** , we **reject the null hypothesis**, meaning **Model A (Random Forest)** significantly outperforms **Model B (SVM)**.

When to Use This Method

- **Use K-Fold CV paired t-test when:**
 - You use **K-fold cross-validation**.
 - You want a **more stable comparison** (reduces variance).
 - You have a **small dataset**.
- **Don't use it when:**
 - You only have **one train-test split** → Use a **paired t-test** instead.

Key Takeaways

- **K-Fold CV paired t-test** provides a **more reliable** comparison than a standard paired t-test.
- **K-Fold CV reduces bias and variance**, making results more stable.

McNemar's Test for Comparing Two Classifiers

- McNemar's test is a **statistical test** used to compare two classifiers based on their misclassification differences on the same dataset.
- It helps determine whether one classifier makes significantly different types of errors compared to another.

When to Use McNemar's Test?

- When comparing two classifiers on paired data (i.e., both models are evaluated on the same test set).
- Best for binary classification problems.
- Suitable when sample size is small and data does not follow a normal distribution.

McNemar's Test

- It analyzes the **disagreements** in classification outcomes using a **contingency table**:

	Model B Correct	Model B Incorrect
Model A Correct	n_{00} (Both correct)	n_{01} (A correct, B wrong)
Model A Incorrect	n_{10} (A wrong, B correct)	n_{11} (Both wrong)

McNemar's Test Statistic

$$\chi^2 = \frac{(|n_{01} - n_{10}| - 1)^2}{n_{01} + n_{10}}$$

- n_{01} = Instances where Model A is correct, but Model B is incorrect.
- n_{10} = Instances where Model A is incorrect, but Model B is correct.

Hypothesis:

- H_0 (Null Hypothesis): The classifiers perform similarly (i.e., they make errors in the same way).
- H_1 (Alternative Hypothesis): The classifiers make significantly different errors.

Decision Rule:

- If p-value < 0.05, reject $H_0 \rightarrow$ Significant difference between classifiers.
- If p-value $\geq$ 0.05, no significant difference.

Example

- Suppose we evaluate **two classifiers (A & B) on 100 samples**, and their predictions result in the following contingency table:

	Model B Correct	Model B Incorrect
Model A Correct	50	10
Model A Incorrect	5	35

- $n_{00} = 50 \rightarrow$ Both models classify 50 samples correctly.
- $n_{01} = 10 \rightarrow$ Model A is correct, but Model B is incorrect.
- $n_{10} = 5 \rightarrow$ Model A is incorrect, but Model B is correct.
- $n_{11} = 35 \rightarrow$ Both models misclassify 35 samples.

McNemar's test focuses on the disagreements: $n_{01}=10, n_{10}=5$

McNemar's Test Statistic Calculation

McNemar's test uses the following formula (for large sample sizes, i.e., when $n_{01} + n_{10} \geq 25$):

$$\chi^2 = \frac{(|n_{01} - n_{10}| - 1)^2}{n_{01} + n_{10}}$$

Substituting values:

$$\chi^2 = \frac{(|10 - 5| - 1)^2}{10 + 5} = \frac{(5 - 1)^2}{15} = \frac{16}{15} = 1.067$$

For **small sample sizes** (when $n_{01} + n_{10} < 25$), we use the **binomial test** instead of the chi-square test.

Step 3: Calculating p-value

We determine the p-value using:

- Chi-square distribution (for large samples).
- Binomial test (for small samples, like this one).

Using the **binomial test**, we calculate the probability of getting at most $\min(n_{01}, n_{10})$ successes out of $n_{01} + n_{10}$, assuming a probability of 0.5 (null hypothesis).

$$p = 2 \times P(X \leq \min(n_{01}, n_{10}))$$

For $n_{01} = 10, n_{10} = 5$:

$$p = 2 \times \sum_{k=0}^5 P(X = k)$$

where $X \sim \text{Binomial}(n = 15, p = 0.5)$.

Using Python, we compute this probability:

Interpretation of Results

- The computed p-value determines if there's a significant difference between the two classifiers.
- If p-value < 0.05 , reject H_0 , meaning the classifiers perform significantly differently.
- If p-value ≥ 0.05 , fail to reject H_0 , meaning the classifiers have no significant difference in performance.

Expected Output:

```
pgsql
```

```
Chi-square statistic (for large samples): 1.067
```

```
Binomial test p-value (for small samples): 0.0625
```

```
McNemar's test p-value (from statsmodels): 0.0625
```

```
Fail to reject the null hypothesis: No significant difference between classifiers.
```

Since $p = 0.0625 > 0.05$, we fail to reject the null hypothesis, meaning that the classifiers do not show a statistically significant difference.

Steps to Calculate the Binomial Test for McNemar's Test

- Define the Null Hypothesis
 - H_0 (Null Hypothesis): The two classifiers have the same error distribution (i.e., their misclassifications are random).
 - This means that the probability of a misclassification being counted in either n_{01} or n_{10} is equal:

$$P(n_{01}) = P(n_{10}) = 0.5$$

Compute the Probability Distribution

- The number of discordant pairs ($n_{01} + n_{10}$) follows a **Binomial distribution**:

$$X \sim \text{Binomial}(n = n_{01} + n_{10}, p = 0.5)$$

- The test calculates the probability of **observing at most** the smaller value of n_{01} and n_{10} :

$$P(X \leq \min(n_{01}, n_{10}))$$

- This probability is computed using the **cumulative probability function (CDF)** of the binomial distribution:

$$P(X \leq k) = \sum_{i=0}^k \binom{n}{i} (0.5)^i (1 - 0.5)^{n-i}$$

where:

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}$$

is the **binomial coefficient**.

Compute the Two-Tailed p-value

Since McNemar's test is a two-tailed test, we double the probability:

$$p = 2 \times P(X \leq \min(n_{01}, n_{10}))$$

Let's assume:

- $n_{01} = 10, n_{10} = 5$
- $n = n_{01} + n_{10} = 10 + 5 = 15$
- Compute:

Thus, the p-value = 0.33, which is greater than 0.05, so we fail to reject the null hypothesis.

$$P(X \leq 5) = \sum_{i=0}^5 \binom{15}{i} (0.5)^{15}$$

Manual Calculation of Binomial Probabilities

i	$\binom{15}{i}$	$P(i) = \binom{15}{i} (0.5)^{15}$
0	1	0.00003
1	15	0.00046
2	105	0.00351
3	455	0.01502
4	1365	0.04578
5	3003	0.10017

$$P(X \leq 5) = 0.00003 + 0.00046 + 0.00351 + 0.01502 + 0.04578 + 0.10017 = 0.1650$$

Final p-value:

$$p = 2 \times 0.1650 = 0.3300$$